```python
from dataclasses import dataclass
import numpy as np
import itertools
import mmh3
from edtrace.file_util import download_file
from edtrace import text, image, link
from lecture_13 import the_pile
from lecture_util import article_link, post_link
from references import dolma_2024, the_pile_2020, dclm_2024

def main():
```

Lecture 14: Data II

Last lecture:

- Live service (e.g., GitHub) → dump/crawl (e.g., GitHub Archive) → processed data (e.g., The Stack)
- Considerations: terms of service, copyright (licenses or fair use)

This lecture:

- Data pipeline: transformation, filtering, deduplication, mixing
- Mid-training + SFT: synthetic data

```python
    # Data pipeline
    transformation()
    filtering()
    deduplication()
    data_mixing()

    # Post-training data
    post_training_data()
```

Summary:

- Filtering: train classifier (language id, quality, toxicity) for what good looks like
- Deduplication: hashing scales to large datasets for fuzzy matching
- Mixing: try mixtures at small scale, extrapolate to optimal mixture and large scale
- Applications: language identification, quality filtering, toxicity filtering
- Post-training data: looks like evaluations, use of synthetic data
- A lot of data work is domain-specific, looking at examples, etc.

```python
def transformation():
```

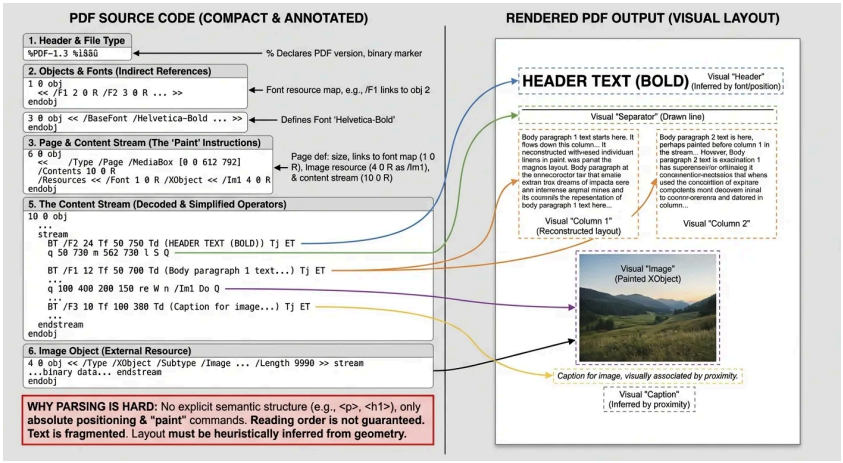
Raw data does not come as text.
It is HTML, PDF (arxiv), or directories (code repositories).

HTML to text (main one):

- Remove boilerplate (e.g., navigation, ads) and extract content
- What about images, tables, etc.?
- Inherently lossy (need to linearize)
- Tools (rule-based): trafilatura, resiliparse, jusText, lynx, etc.
- Accuracy matters: [Li+ 2024]

Text Extraction	CORE	EXTENDED
`resiliparse`	24.1	**13.4**
`trafilatura`	**24.5**	12.5
WET files	20.7	12.2

FinePDFs [post]



- Source: Common Crawl
- Recrawl truncated PDFs (since they are big)
- OCR (RolmOCR) using a VLM or Docling (make these run fast)
- Lots of cleanup and filtering
- A lot of layout information is missing

```
def filtering():
```

Algorithmic building block:

- Given some **target data** T and lots of **raw data** R, find subset T' of R similar to T.

Applications:

- Language identification (English versus rest)
- Quality filtering (high quality versus low quality)
- Toxicity filtering (non-toxic versus toxic)

Desiderata for filtering algorithm:

- Generalize from the target data (want T and T' to be different)
- Extremely fast (have to run it on R, which is huge)

Survey paper on data selection [Albalak+ 2024]

General framework: Given target T and raw R, find subset of R similar to T
1. Estimate some model based on R and T and derive a scoring function
2. Keep examples in R based on their score

Types of classifiers:

- Generative model of T (KenLM): score(x) = p_T(x)
- Simple classifier (fastText): score(x) = p(T | x)

To use: keep examples x with score(x) >= threshold (stochastically)

Model-based filtering?

- Some deliberately do not use model-based filtering (C4, Gopher, RefinedWeb, FineWeb, Dolma)
- Some use model-based filtering (GPT-3, LLaMA, DCLM) [becoming the norm]

Language identification:

- Goal: find text of a specific language (e.g., English)
- fastText language identification [article]
- Off-the-shelf classifier
- Supports 176 languages
- Trained on multilingual sites: Wikipedia, Tatoeba (translation site) and SETimes (Southeast European news)
- Dolma keeps pages with p(English) >= 0.5 [Soldaini+ 2024]

OpenMathText [Paster+ 2023]

- Goal: curate large corpus of mathematical text from CommonCrawl
- Use rules to filter (e.g., contains latex commands)
- KenLM trained on ProofPile, keep if perplexity < 15000
- Trained fastText classifier to predict mathematical writing, threshold is 0.17 if math, 0.8 if no math
- Result: produced 14.7B tokens, used to train 1.4B models that do better than models trained on 20x data

GPT-3 [Brown+ 2020]

- Positives: samples from {Wikipedia, WebText2, Books1, Books2}
- Negatives: samples from CommonCrawl

Train linear classifier based on word features [article]

Keep documents stochastically based on score

```
def keep_document(score: float) -> bool:
    return np.random.pareto(9) > 1 - score
```

LLaMA/RedPajama [Touvron+ 2023]

- Positives: samples from pages **referenced** by Wikipedia
- Negatives: samples from CommonCrawl
- Keep documents that are classified positive

phi-1 [Gunasekar+ 2023]

- Philosophy: really high quality data (textbooks) to train a small model (1.5B)
- Includes synthetic data from GPT 3.5 (later: GPT-4) and filtered data

```
R = "Python subset of the Stack"   # Raw data
prompt = "determine its educational value for a student whose goal is to learn basic coding concepts"
T = "Use GPT-4 with this prompt to classify 100K subset of R to get positive examples"
```

- Train random forest classifier on T using output embedding from pretrained codegen model
- Select data from R that is classified positive by the classifier

Result on HumanEval:

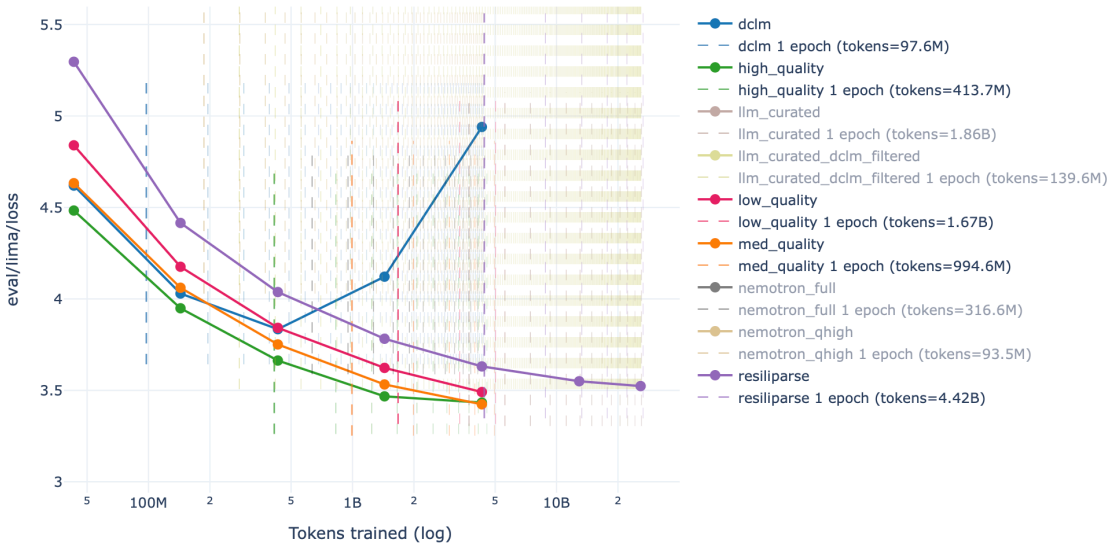
- Train 1.3B LM on Python subset of The Stack (performance: 12.19% after 96K steps)
- Train 1.3B LM on new filtered subset (performance: 17.68% after 36K steps) - better!

Toxicity filtering in Dolma [Soldaini+ 2024]

- Dataset: Jigsaw Toxic Comments dataset (2018) [dataset]
- Project goal: help people have better discussions online [article]
- Data: comments on Wikipedia talk page annotated with {toxic, severe_toxic, obscene, threat, insult, identity_hate}

Scale-dependent effects of filtering:

- No single optimal threshold for filtering
- If training for longer, want more (lower quality) data
- If training for shorter, want less (higher quality) data



Summary:

- Filtering is critical for building a good model
- Recipe: define target data (what good looks like), extrapolate to raw data

```
def deduplication():
```

Two types of duplicates:

- Exact duplicates (mirror sites, GitHub forks) [Gutenberg mirrors]
- Near duplicates: same text differing by a few tokens

Examples of near duplicates:

- Terms of service and licenses [MIT license]
- Formulaic writing (copy/pasted or generated from a template)

Dataset	Example	Near-Duplicate Example
Wiki-40B	\n_START_ARTICLE_\nHum Award for Most Impactful Character \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]	\n_START_ARTICLE_\nHum Award for Best Actor in a Negative Role \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]
LM1B	I left for California in 1979 and tracked Cleveland 's changes on trips back to visit my sisters .	I left for California in 1979 , and tracked Cleveland 's changes on trips back to visit my sisters .
C4	Affordable and convenient holiday flights take off from your departure country, "Canada". From May 2019 to October 2019, Condor flights to your dream destination will be roughly 6 a week! Book your Halifax (YHZ) - Basel (BSL) flight now, and look forward to your "Switzerland" destination!	Affordable and convenient holiday flights take off from your departure country, "USA". From April 2019 to October 2019, Condor flights to your dream destination will be roughly 7 a week! Book your Maui Kahului (OGG) - Dubrovnik (DBV) flight now, and look forward to your "Croatia" destination!

- Minor formatting differences in copy/pasting

Product description repeated 61,036 times in C4

'"by combining fantastic ideas, interesting arrangements, and follow the current trends in the field of that make you more inspired and give artistic touches. We'd be honored if you can apply some or all of these design in your wedding. believe me, brilliant ideas would be perfect if it can be applied in real and make the people around you amazed!

[example page]

Deduplication training data makes language models better [Lee+ 2021]

- Train more efficiently (because have fewer tokens)
- Avoid memorization (can mitigate copyright, privacy concerns)

Design space:

1. What is an item (sentence, paragraph, document)?
2. How to match (exact match, existence of common subitem, fraction of common subitems)?
3. What action to take (remove all, remove all but one)?

Key challenge:

- Deduplication is fundamentally about comparing items to other items
- Need linear time algorithms to scale

```
hash_functions()
exact_deduplication()
jaccard_minhash()
locality_sensitive_hashing()
```

```
def hash_functions():
```

- Hash function h maps item to a hash value (integer or string)
- Hash value much smaller than item
- Hash collision: h(x) = h(y) for x ≠ y

Tradeoff between efficiency and collision resistance [article]

- Cryptographic hash functions (SHA-256): collision resistant, slow (used in bitcoin)
- DJB2, MurmurHash, CityHash: not collision resistant, fast (used for hash tables)

We will use MurmurHash:

```
h = mmh3.hash("hello")
```

```
def exact_deduplication():
```

Simple example

1. Item: string
2. How to match: exact match
3. Action: remove all but one

```
# Original items
items = ["Hello!", "hello", "hello there", "hello", "hi", "bye"]

# Compute hash -> list of items with that hash
hash_items = itertools.groupby(sorted(items, key=mmh3.hash), key=mmh3.hash)

# Keep one item from each group
deduped_items = [next(group) for h, group in hash_items]
```

- Pro: simple, clear semantics, high precision
- Con: does not deduplicate near duplicates
- This code is written in a MapReduce way, can easily parallelize and scale

C4 [Raffel+ 2019]

1. Item: 3-sentence spans
2. How to match: use exact match
3. Action: remove all but one

Warning: when a 3-sentence span is removed from the middle of a document, the resulting document might not be coherent

```
def jaccard_minhash():
```

Let's now look at approximate set membership.
First we need a similarity measure.

Jaccard similarity

Definition: Jaccard(A, B) = |A intersect B| / |A union B|

```
A = {"1", "2", "3", "4"}
B = {"1", "2", "3", "5"}

def compute_jaccard(A, B):
    intersection = len(A & B)
    union = len(A | B)
    return intersection / union
jaccard = compute_jaccard(A, B)
```

Definition: two documents are **near duplicates** if their Jaccard similarity >= threshold

Algorithmic challenge: find near duplicates in linear time

MinHash

MinHash: a random hash function h so that Pr[h(A) = h(B)] = Jaccard(A, B)

Normally, you want different items to hash to different hashes
...but here, you want collision probability to depend on similarity

```
def minhash(S: set[str], seed: int):
    return min(mmh3.hash(x, seed) for x in S)
```

Characteristic matrix representation:

```
item | A | B
1    | 1 | 1
2    | 1 | 1
3    | 1 | 1
4    | 1 | 0
5    | 0 | 1
```

Random hash function induces a permutation over items
Look at which item is first in A and which item is first in B.
Each item has the same probability as being first (min)

- If 1, 2, 3 is first, then first in A = first in B.
- If 4, 5 is first, then first in A ≠ first in B.

```
# Verify MinHash approximates Jaccard as advertised
n = 100  # Generate this many random hash functions
matches = [minhash(A, seed) == minhash(B, seed) for seed in range(n)]
estimated_jaccard = len([m for m in matches if m]) / len(matches)
assert abs(estimated_jaccard - jaccard) < 0.01
```

Now we can hash our items, but a collision doesn't tell us Jaccard(A, B) > threshold.

```
def locality_sensitive_hashing():
```

Locality sensitive hashing (LSH) [book chapter]

Suppose we hash examples with just one MinHash function
P[A and B collide] = Jaccard(A, B)
On average, more similar items will collide, but very stochastic...

Goal: have A and B collide if Jaccard(A, B) > threshold
We have to somehow sharpen the probabilities...

Solution: use n hash functions
Break up into b bands of r hash functions each (n = b * r)

```
n = 12      # Number of hash functions
b = 3       # Number of bands
r = 4       # Number of hash functions per band
```

Hash functions:

```
h1 h2 h3 h4  |  h5 h6 h7 h8  |  h9 h10 h11 h12
```

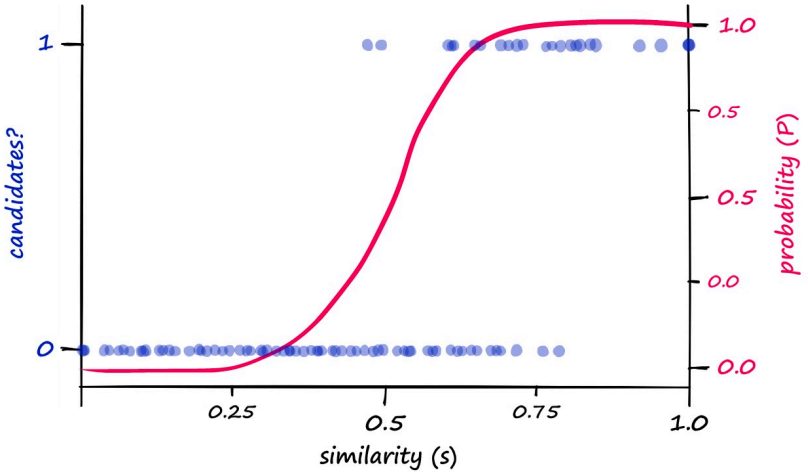
Key: A and B collide if for *some* band, *all* its hash functions return same value
As we will see, the and-or structure of the bands sharpens the threshold

Given Jaccard(A, B), what is the probability that A and B collide?

```
def get_prob_collision(sim, b, r):
    prob_match = sim ** r                       # Probability that a fixed band matches
    prob_collision = 1 - (1 - prob_match) ** b   # Probability that some band matches
    return prob_collision
```

Example

```
prob_collision = get_prob_collision(sim=0.8, b=5, r=10)
```



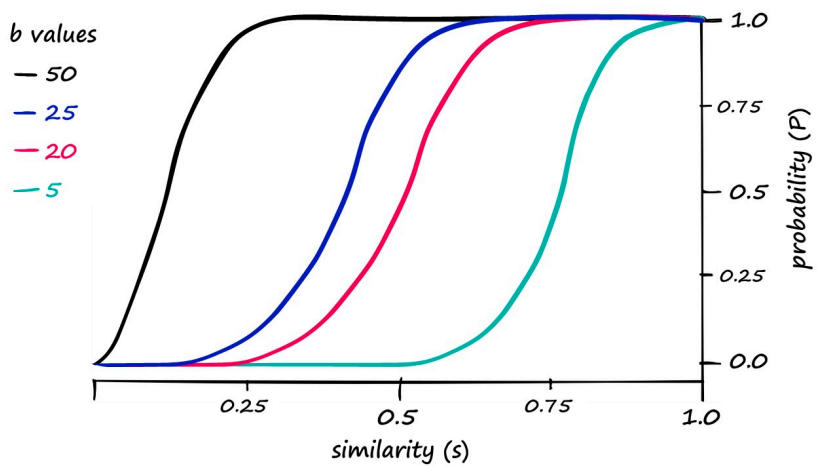

```
sims = [0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.98]
probs = {sim: get_prob_collision(sim=sim, b=10, r=10) for sim in sims}
```

Increasing r sharpens the threshold and moves the curve to the right (harder to match)

```
probs = {sim: get_prob_collision(sim=sim, b=10, r=20) for sim in sims}
```

Increasing b moves the curve to the left (easier to match)

```
probs = {sim: get_prob_collision(sim=sim, b=20, r=20) for sim in sims}
```



Example setting [Lee+ 2021]: n = 9000, b = 20, r = 450

```
b = 20
```

```
r = 450
```

What is the threshold (where the phase transition happens)?

```
threshold = (1 / b) ** (1 / r)
```

Probability that a fixed band matches:

```
prob_match = (1 / b)
```

Probability that A and B collide is a constant (≈ 1-1/e):

```
prob_collision = 1 - (1 - 1 / b) ** b
```

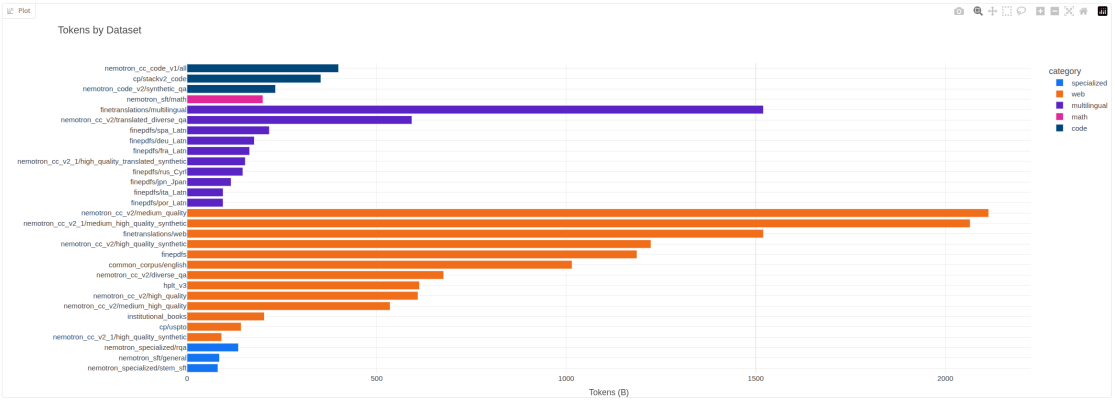
```
def billion(x):
    return x * 10**9

def trillion(x):
    return x * 10**12


def data_mixing():
```

Recall that language models are trained on multiple data sources.

Datasets in Marin: [token viewer]



The Pile [Gao+ 2020]

Component	Raw Size	Weight	Epochs	Effective Size	Mean Document Size
Pile-CC	227.12 GiB	18.11%	1.0	227.12 GiB	4.33 KiB
PubMed Central	90.27 GiB	14.40%	2.0	180.55 GiB	30.55 KiB
Books3†	100.96 GiB	12.07%	1.5	151.44 GiB	538.36 KiB
OpenWebText2	62.77 GiB	10.01%	2.0	125.54 GiB	3.85 KiB
ArXiv	56.21 GiB	8.96%	2.0	112.42 GiB	46.61 KiB
Github	95.16 GiB	7.59%	1.0	95.16 GiB	5.25 KiB
FreeLaw	51.15 GiB	6.12%	1.5	76.73 GiB	15.06 KiB
Stack Exchange	32.20 GiB	5.13%	2.0	64.39 GiB	2.16 KiB
USPTO Backgrounds	22.90 GiB	3.65%	2.0	45.81 GiB	4.08 KiB
PubMed Abstracts	19.26 GiB	3.07%	2.0	38.53 GiB	1.30 KiB
Gutenberg (PG-19)†	10.88 GiB	2.17%	2.5	27.19 GiB	398.73 KiB
OpenSubtitles†	12.98 GiB	1.55%	1.5	19.47 GiB	30.48 KiB
Wikipedia (en)†	6.38 GiB	1.53%	3.0	19.13 GiB	1.11 KiB
DM Mathematics†	7.75 GiB	1.24%	2.0	15.49 GiB	8.00 KiB
Ubuntu IRC	5.52 GiB	0.88%	2.0	11.03 GiB	545.48 KiB
BookCorpus2	6.30 GiB	0.75%	1.5	9.45 GiB	369.87 KiB
EuroParl†	4.59 GiB	0.73%	2.0	9.17 GiB	68.87 KiB
HackerNews	3.90 GiB	0.62%	2.0	7.80 GiB	4.92 KiB
YoutubeSubtitles	3.73 GiB	0.60%	2.0	7.47 GiB	22.55 KiB
PhilPapers	2.38 GiB	0.38%	2.0	4.76 GiB	73.37 KiB
NIH ExPorter	1.89 GiB	0.30%	2.0	3.79 GiB	2.11 KiB
Enron Emails†	0.88 GiB	0.14%	2.0	1.76 GiB	1.78 KiB
The Pile	**825.18 GiB**			**1254.20 GiB**	**5.91 KiB**

Key question: what distribution over the data sources should we use?

Example:

```
sources = {"Wikipedia", "CC", "GitHub"}
p = {"Wikipedia": 0.3, "CC": 0.5, "GitHub": 0.2}  # One possible data mixture
```

Baselines:

- Vibes: set p(s) manually based on intuition (quite common)
- Uniform sampling: sample uniformly (p(s) $\propto$ 1)
- Proportional mixing: sample proportional to the number of tokens in a source (p(s) $\propto$ num_tokens(s))

Intuition: should upweight higher quality sources

However...

1. We want to ensure diversity (e.g., across incomparable sources: literature, code, papers)
2. Each source is finite, so if put too much weight on a small source, then need to epoch over it

This last point is important and a bit subtle.

Example:

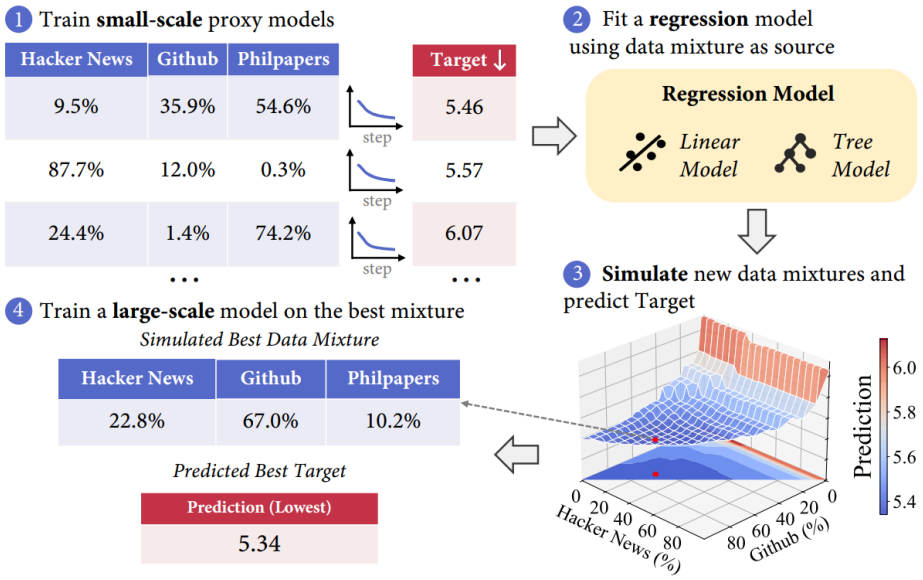
```
source_token_counts = {
    "low": trillion(10),  # 10T tokens (abundant)
    "high": billion(10),  # 10B tokens (scarce)
}
p = {"low": 0.5, "high": 0.5}  # Naive data mixture
train_tokens = trillion(1)  # Train for 1T tokens
low_num_epochs = (p["low"] * train_tokens) / source_token_counts["low"]
high_num_epochs = (p["high"] * train_tokens) / source_token_counts["high"]
```

50x epochs on high quality data...can lead to overfitting!

UniMax [Chung+ 2023]

- Setting: balancing different languages for multilingual models
- Previous work: between uniform and proportional mixing (p(s) $\propto$ num_tokens(s)^α for α in [0, 1])
- Idea: sample sources uniformly but with a hard **cap** C on number of epochs for any source
- Specifically, p(s) * num_training_tokens ≤ C for all sources s

Regression-based mixing [Liu+ 2024][Chen+ 2026]



- Define distribution over mixtures p (e.g., Dirichlet)
- Define regression method (e.g., linear, gradient boosted trees)
- Define target based on downstream evals (careful not to overfit!)
- Discrepancy between small and large scale (tradeoff cost and accuracy)

Design Choice	RegMix (Liu et al., 2025a)	DML (Ye et al., 2025)	AutoScale (Kang et al., 2025)	BiMix (Ge et al., 2025b)	ADMIRE-BayesOpt (Chen et al., 2025b)	CLIMB (Diao et al., 2025)	OLMIXBASE (Algorithm 1)
Swarm Construction							
Proxy model size	1M	70, 160, 305, 410M	Target	280M	1M, 60M	350M	30M
Swarm size (vs m domains)	512 (m = 17)	20 (m = 7)	$2m + 1$	4	101 (m = 17)	112 (m = 21)	$3(m + 1)$
Swarm distribution	Dirichlet with natural prior	Exponential grid	Exponential grid	Entropy-weighted	Dynamic	Dirichlet with natural prior	Dirichlet with natural prior (sparse for topics, dense for sources)
Regression Model							
Regression model family	LightGBM	Log-Linear	Power Law	Power Law	Gaussian Process	LightGBM	Log-Linear
Regression granularity	Aggregated	Aggregated	Per-Task	Per-Task	Aggregated	Aggregated	Per-Task
Mixture Optimization							
Data repetition constraints	No	No	No	No	No	No	Yes
Optimization solver	Search	Search	Gradient Descent	Exact Solver	Search	Search	Exact Solver with KL reg.

Hope 1: regression model is accurate at minimizer 🙏

Hope 2: optimal data mixtures transfer from small to large scale 🙏

Hold on. There's at least one scale-dependent effect:

```
source_token_counts = {
    "low": trillion(10),  # 10T tokens (abundant)
    "high": billion(10),  # 10B tokens (scarce)
}
```

- If train small models on low token counts:

```
p = {"low": 0.1, "high": 0.9}  # More mass on high quality data
```

- But if train large model on this mixture, we will epoch a ton on high quality data and overfit!

Simulated epoching [Held+ 2025]

- General idea: make small scale look like large scale (general theme of this course)
- Instantiation: downsample all sources proportionally

```
small_run_tokens = billion(10)
large_run_tokens = trillion(1)
ratio = small_run_tokens / large_run_tokens
downsampled_source_token_counts = {s: count * ratio for s, count in source_token_counts.items()}
```

- In this downsampled mixture, models that epoch too much won't look good.
- So the optimum will be more balanced.

```
p = {"low": 0.7, "high": 0.3}  # More mass on high quality data
```

Summary:

- Problem: how to weight different data sources (e.g., Wikipedia, general, code)
- Regression-based mixing: estimate mixture → loss at small scale, optimize (analogous to scaling laws)
- Important consideration: epoching and overfitting (solution: cap or simulated)

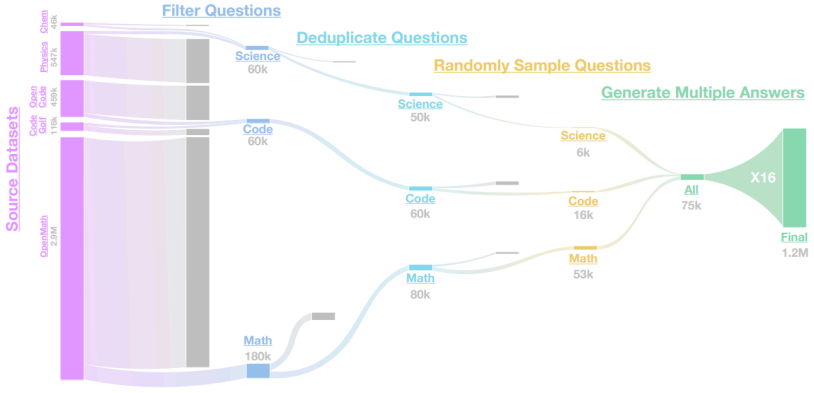
```
def post_training_data():
```

Recipe:

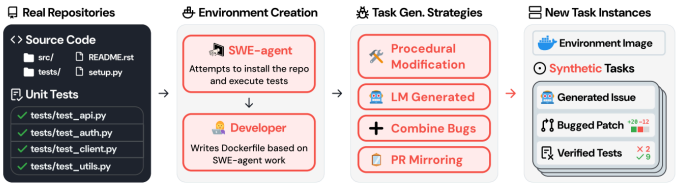
1. Define a set of environments
2. Define a set of tasks / prompts
3. Collect responses from a strong model (teacher)

OpenThoughts [Guha+ 2025]

- 1.2M examples using QwQ-32B as a teacher
- Questions come from 27 human and synthetic sources (e.g., StackExchange, NuminaMath, Chemistry)
  - **StackExchange CodeGolf** (Number of Questions: 85.9K): A StackExchange forum of coding puzzles, specifically aimed at solutions with the least number of characters possible.
  - **OpenCodeReasoning** (Number of Questions: 459K) A large reasoning-based synthetic dataset to date for coding, comprises 735,255 samples in Python across 28,319 unique competitive programming questions
  - **cognitivecomputations/dolphin-coder** (Number of Questions: 101K): Synthetic questions evolved from LeetCode questions.
  - **m-a-p/CodeFeedback-Filtered-Instruction** (Number of Questions: 150K): Mixture of synthetic and real coding questions filtered by an LLM.
  - **KodCode/KodCode-V1** (Number of Questions: 384K): Fully synthetic and diverse coding dataset with questions ranging from algorithmic to package specific knowledge.
  - **Multilingual-Multimodal-NLP/McEval-Instruct** (Number of Questions: 35.8K): Multilingual code dataset on code-understanding, completion, and generation.
  - **christopher/rosetta-code** (Number of Questions: 75.4K): Multilingual code dataset on basic coding exercises.
  - **glaiveai/glaive-code-assistant-v3** (Number of Questions: 946K): Code problems and solutions generated using Glaive's synthetic data generation platform.
  - **StackExchange CodeReview** (Number of Questions: 183K): Code review questions from codereview.meta.stackexchange.com.
  - **prithivMLmods/Coder-Stat** (Number of Questions: 41.9K): Coding dataset for the analysis of coding patterns, error types, and performance metrics. We transform code and an associated error into a question for the LLM to solve using GPT-4o-mini with the prompt in Figure 16.
  - **OpenCoder-LLM/opc-sft-stage2**: A mixture of synthetic python questions generated from python documentation, educational material and more. We use both OpenCoder-LLM/opc-sft-stage2 and OpenCoder-LLM/opc-sft-stage1. Specifically, we use the package_instruct subset of OpenCoder-LLM/opc-sft-stage2 and the filtered_infinity_instruct, largescale_diverse_instruct, and realuser_instruct subsets of OpenCoder-LLM/opc-sft-stage1.
- Sampling multiple (16) responses per prompt is helpful
- Better models aren't necessarily better teachers: QwQ-32B is a better teacher than DeepSeek-R1
- Answer filtering wasn't helpful
- Smaller high quality sources (e.g., OpenMath-2-Math) is better than large diverse sources



SWE-smith [Yang+ 2025]



- Given a repository, use LM to generate tasks (introduce bugs with LM)
- 128 GitHub repositories yields 50K tasks

SWE-Zero[Ludwig+ 2026]

- SWE tasks have heavy dependencies (unlike math or coding contests)
- Setting up thousands of Docker images is an infrastructural nightmare
- Observation: strong models can solve many tasks without execution feedback

Model	Execution	SWE-bench (V)	SWE-bench (M)
MiniMax-M2.5	×	69.5	57.2
	✓	80.2	74.1
Qwen3-Coder-Next	×	56.9	50.7
	✓	71.3	64.3
Qwen3-Coder-480B-A35B-Instruct	×	59.4	44.3
	✓	69.6	54.7
SWE-HERO-32B (Ours)	×	57.7	42.2
	✓	62.2	44.1

Key: strong models have internal "world model" of code semantics

- SWE-Zero: 300K agent trajectories that don't require repository-specific execution
- 150K GitHub PRs
- OpenHands scaffold, remove future git commits to prevent "git hacking" by agent

Standard OpenHands Setup (Execution-Based)

System: You are OpenHands agent, a helpful AI assistant that can interact with a computer to solve tasks. The development Python environment is already set up for you.

Problem Solving Workflow:

- **Exploration:** Thoroughly explore relevant files and context.
- **Analysis:** Consider multiple approaches and select the most promising one.
- **Testing:** Create tests to verify issues; delete temporary files after confirmation.
- **Implementation:** Make focused, minimal changes to address the problem.
- **Verification:** Thoroughly test implementation, including edge cases.

User Instruction Phases:
Phase 1: Reading → Phase 2: Running → Phase 3: Exploration → Phase 4: Test Creation → Phase 5: Fix Analysis → Phase 6: Implementation → Phase 7: Verification → Phase 8: Final Review.

SWE-Zero OpenHands Setup (Execution-Free)

System: You are OpenHands agent... The development environment is **unavailable**. You **CANNOT RUN PYTHON CODE** for any purpose. Do not use Python to check your work.

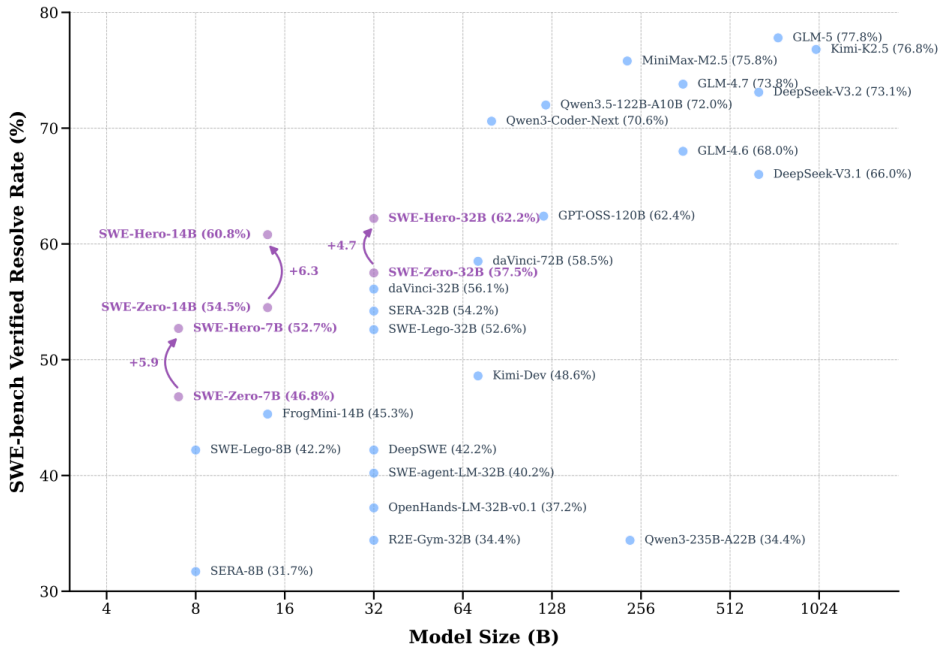
Problem Solving Workflow:

- **Exploration:** Thoroughly explore files and understand context.
- **Analysis:** Consider multiple approaches (Static Analysis).
- **Implementation:** Make focused, minimal changes. **Do not write or execute any tests.**

Prohibited Bash Commands:
`python` (including `-c`/`-m` options), `pytest`, `mypy`, `pip`, `apt`, `apt-get`.

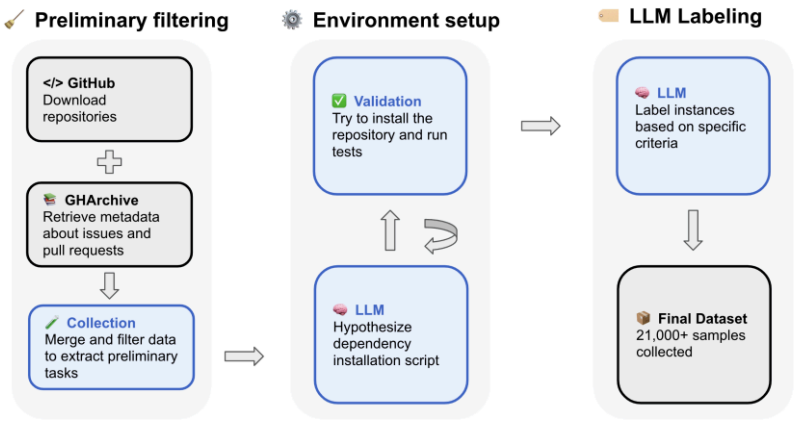
User Instruction Phases:
Phase 1: Reading → Phase 2: Exploration → Phase 3: Fix Analysis → Phase 4: Implementation → Phase 5: Final Review.

- Distilled from Qwen3-Coder-480B + filtering (try to execute anyway)
- SWE-Hero: 13K agent trajectories that do require execution feedback



SWE-rebench [Badertdinov+ 2025]

- 21K interactive Python SWE tasks from 3.4K GitHub repositories
- 450K PRs from GitHub and GitHub Archive
- Used Qwen 2.5-72B-Instruct to install dependencies and assess PR quality



SWE-ZERO-12M-trajectories [data]

- Scale SWE-Zero up to 12M agent trajectories
- Used the SWE-rebench-v2 tasks (32K executable tasks + 120K nonexecutable tasks)
- Ran mini-coder-1.7b (very small model, 50.4 pass@100), mini-swe-agent scaffold
- Example

Summary:

- Generating prompts: fully-synthetic, semi-synthetic (real environment + synthetic tasks), real (GitHub PRs)
- Responses: from capable models (that are also good teachers)
- Code environments are painful
- Lots of filtering and other details

```
if __name__ == "__main__":
    main()
```